

Chapter 1

Reinforcement Learning

This chapter introduces the fundamental concepts of deep learning, providing the necessary foundation for the ideas developed throughout the rest of this manuscript. This chapter can be seen as a synthesis of the reference work *Deep Learning* [?]. Its structure is as follows:

Its structure is as follows:

Section 1.1: An overview of the milestones that led to the emergence of reinforcement learning as a unified field.

Section 1.2: A formalization of the objective underlying reinforcement learning algorithms.

Section 1.3: An explanation of how policies are progressively refined through interaction with the environment to approach the optimal policy.

1.1 Historical Developments

Reinforcement learning is today presented as a major field within artificial intelligence. However, it originates from several research traditions that progressively converged toward a common framework. In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

1.1.1 Behavioral foundations (1900–1950)

The origins of reinforcement learning can be traced back to 20th-century animal psychology studies [?]. These studies aimed to understand how behaviors are formed and modified. The central idea emerging from this work is that learning is driven by the consequences of actions. Behaviors followed by positive outcomes are more likely to be repeated, whereas those associated with negative outcomes tend to gradually decrease in frequency.

This perspective was later developed within behaviorism. In this school of psychology, rewards and punishments are used to gradually shape behavior through repeated interaction with the environment [?]. The term “reinforcement” in the context of learning comes from this school of thought, where it refers precisely to the process of strengthening a behavior through its consequences.

This idea led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a “pleasure–pain system” [?]. Although these studies did not involve formal mathematical models, they introduced the central idea of reinforcement learning, trial-and-error learning guided by environmental feedback.

1.1.2 Sequential decision-making (1950–1970)

A second foundation of reinforcement learning comes from optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. In this context, the decision-making entity is called an agent. The agent interacts with the environment by selecting actions. The function that maps an observed state to the chosen action is called a policy. The objective is to find an optimal policy, in the sense that it maximizes the expected cumulative reward over time.

This framework led to the formalization of Markov Decision Processes (MDPs), which provide a mathematical model for sequential decision-making under uncertainty [?]. MDPs describe a decision-maker interacting with an environment through a set of states, actions, rewards, and transition rules. The earliest method proposed to solve MDPs is dynamic programming [?]. It proceeds by breaking down a complex decision problem into smaller, more manageable subproblems, solving each one, and combining the results to obtain an overall solution. However, dynamic programming requires a complete and explicit description of the environment, which quickly becomes impractical for all but the smallest problems.

1.1.3 Emergence of reinforcement learning (1970–2000)

In order to address this issue, temporal-difference (TD) learning methods were introduced [?]. This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations.

Building on these ideas, the notion of value functions emerged as a way to estimate the long-term benefit of being in a given state or taking a given action. These functions assign a numerical score to states or state-action pairs, reflecting the expected cumulative reward an agent can obtain from that point onward. They thus provide a compact way to evaluate and compare different courses of action. This line of work culminated in Q-learning, which showed that optimal action-value functions can be learned directly from interaction with the environment, without requiring a model of its dynamics [?, ?].

Shortly before the turn of the millennium, the publication of the foundational reference work [?] contributed to the formalization of reinforcement learning as a coherent field. From that point onward, the field can be considered well established and widely recognized as a distinct area of research.

74 1.1.4 Consolidation of reinforcement learning (2000–2013)

75 A limitation of early algorithms such as Q-learning is their reliance on explicit
 76 state-action tables. Although these methods remove the need to know the
 77 full dynamics of the MDP, they still require storing a value for each possible
 78 state-action pair. As a result, the agent must effectively visit and maintain
 79 estimates for a potentially enormous number of states in the environment. This
 80 requirement becomes infeasible in large-scale MDPs, where the state space is
 81 too large to be exhaustively explored or stored in memory.

82 To address this issue, research shifted toward more general representations
 83 of value functions and policies. Instead of maintaining explicit tables, function
 84 approximation methods were introduced. These allowed value functions and
 85 policies to be represented using parametric models such as linear approximators
 86 or perceptron-like architectures [?, ?]. This change enabled generalization across
 87 states, reducing the dependence on exhaustive exploration of the state space.
 88 The introduction of Deep Q-Networks (DQN), which demonstrated that deep
 89 Neural Network (NN) could be used to approximate value functions at scale [?].

90 1.1.5 Deep reinforcement learning (2013–2026)

91 The introduction of deep learning into reinforcement learning marked a major
 92 turning point in the field. A series of algorithms significantly improved scalability.
 93 Deep Deterministic Policy Gradient (DDPG) extended actor-critic methods to
 94 continuous action spaces [?]. Proximal Policy Optimization (PPO) later became
 95 a standard algorithm in reinforcement learning due to its simplicity, stability,
 96 and ability to perform in both continuous and discrete action spaces [?].

97 These advances established a practical foundation for large-scale policy
 98 optimization. Notably, AlphaZero demonstrated that the combination of deep
 99 NN, reinforcement learning, and tree search could reach superhuman performance
 100 in the game of Go [?]. This success was later extended to even more complex
 101 real-time strategy environments such as StarCraft II, where agents must handle
 102 long-term planning, partial observability, and large action spaces [?]. Similarly,
 103 in Dota 2, large-scale multi-agent reinforcement learning systems such as OpenAI
 104 Five showed that self-play combined with deep policy optimization can achieve
 105 professional-level performance in highly dynamic and stochastic environments
 106 [?].

107 More recently, reinforcement learning has expanded beyond its traditional
 108 application domains, such as robotics, video games, and combinatorial optimiza-
 109 tion. With the emergence of increasingly agentic views of large language models,
 110 it has become possible to train these models to perform specific tasks using
 111 reinforcement learning-based techniques. In particular, reinforcement learning
 112 from human feedback has become a method for aligning large language models
 113 with human preferences. It does so by formalizing the problem as an MDP and
 114 using reinforcement learning to solve it [?]. Similarly, DeepSeekMath shows
 115 how reinforcement learning can enhance the mathematical reasoning abilities
 116 of large language models [?]. It demonstrates that an artificial agent can learn
 117 to reason in natural language in order to accomplish a given task. The skills
 118 acquired during this learning process also transfer to other similar tasks, leading
 119 to improved overall performance. For instance, training a large language model
 120 with reinforcement learning on mathematical problem-solving can also enhance

121 its programming capabilities.

122 Originally inspired by biological learning processes, reinforcement learning is
 123 now closer than ever to this initial intuition. Agents learn through interaction,
 124 adapt via trial and error, and progressively improve their behavior based on
 125 experience. This behavior is governed by deep neural architectures, which are the
 126 closest computational analog to organic brains. More than ever, reinforcement
 127 learning stands as one of the core paradigms for achieving artificial general
 128 intelligence [?].

129 1.2 Objective

130 Now that the main developments shaping reinforcement learning have been
 131 outlined, we proceed to the formal definition of its objective. In essence, rein-
 132 forcement learning aims to build a system that makes sequential decisions in
 133 order to maximize a cumulative reward signal over the long term. The purpose
 134 of this section is to translate this intuitive goal into a rigorous mathematical
 135 framework. To this end, we proceed step by step:

- 136 1. We define how problems are modeled using Markov Decision Processes in
 137 Subsection 1.2.1.
- 138 2. We introduce the notion of policy, which formalizes the agent’s behavior,
 139 in Subsection 1.2.2.
- 140 3. We describe how the interaction between the agent and the environment is
 141 characterized through trajectories in Subsection 1.2.3.
- 142 4. We present how policies are evaluated using value functions in Subsec-
 143 tion 1.2.4.
- 144 5. We define the optimal policy in Subsection 1.2.5, which is the ultimate
 145 goal of reinforcement learning.

146 1.2.1 Markov Decision Processes

147 The RL framework is grounded in the agent paradigm: the decision-making
 148 entity is called the agent. The agent perceives its environment, processes the
 149 information it receives, and acts upon the environment to influence future
 150 outcomes. To specify both the task and the agent’s capacity to perceive and act,
 151 we model the problem as a Markov Decision Process (MDP).

152 Before providing a more detailed definition, it is useful to clarify the scope of
 153 the MDP problems considered in this manuscript. The goal of this subsection is
 154 to introduce the notations and formalisms as they are used in modern RL practice.
 155 Other formulations of MDPs exist that are better suited, for instance, to proving
 156 convergence guarantees. Our focus here is to represent reinforcement learning as
 157 it is applied empirically, even if this means sacrificing some mathematical proof
 158 capability. We therefore restrict our analysis to the following configuration:

- 159 • We consider the single-agent learning setting. If any other agents are
 160 present in the environment, they are treated as part of the environment
 161 dynamics.

- 162 • We focus on finite-horizon MDPs. Each interaction sequence between the
163 MDP and the policy has a defined beginning and a defined end. Between
164 these two points, the number of interaction steps may vary depending on
165 the situation, but it always remains finite.
- 166 • We adopt the partially observable framework, where the agent does not have
167 direct access to the true state of the system, but instead relies on limited
168 observations that provide only partial information about the environment.

169 The resulting framework can be formally described as a single-agent finite-
170 horizon partially observable markov decision process. For simplicity, we will refer
171 to this setting as an “MDP” throughout this manuscript. A MDP is defined by
172 the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$:

- 173 • $\mathcal{S}$: The set of states, representing all possible configurations of the environ-
174 ment. A state belonging to this set is denoted by $s \in \mathcal{S}$. Among these, we
175 distinguish two special subsets: initial states, from which an interaction
176 can begin, and terminal states, in which the interaction ends. The set of
177 initial states is denoted by $\mathcal{S}^i$, with an initial state written as $s^i \in \mathcal{S}^i$. The
178 set of terminal states is denoted by $\mathcal{S}^t$, with a terminal state written as
179 $s^t \in \mathcal{S}^t$.
- 180 • $\mathcal{O}$: The set of observations that the agent can perceive from the environ-
181 ment. An observation belonging to this set is denoted by $o \in \mathcal{O}$. Unlike the
182 true state $s \in \mathcal{S}$, which is hidden from the agent, the observation $o \in \mathcal{O}$
183 provides partial information about the current state.
- 184 • $\mathcal{A}$: The set of actions that the agent may execute. An action belonging to
185 this set is denoted by $a \in \mathcal{A}$. Actions influence the dynamics of the MDP,
186 thereby shaping its future course. They represent the agent’s capacity to
187 act upon its environment.
- 188 • $\mathcal{R}$: The set of rewards that the agent can receive from the environment. A
189 reward belonging to this set is denoted by $r \in \mathcal{R}$. $\mathcal{R}$ is a subset of $\mathbb{R}$. The
190 agent’s goal is to maximize the cumulative sum of these rewards over time.
- 191 • p : The transition function, which defines the dynamics of the environment.
192 Given a state s and an action a , this function defines a probability distribu-
193 tion over the joint outcome consisting of the next state s' , the observation
194 o , and the reward r . We denote this function by p , and it is defined as:
195 $p : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{P}(\mathcal{S} \times \mathcal{O} \times \mathcal{R})$. The notation $(s', o, r) \sim p(s, a)$ means that
196 the tuple (s', o, r) is sampled from the distribution induced by $p(\cdot \mid s, a)$.
197 The notation $p(s', o, r \mid s, a)$ denotes the probability of observing the tuple
198 (s', o, r) under the distribution $p(\cdot \mid s, a)$.

199 1.2.2 Policy

200 The agent’s decision-making capability is modeled by a policy function. The
201 policy defines the stochastic mapping between observations $o \in \mathcal{O}$ and actions
202 $a \in \mathcal{A}$. We denote a policy function by π , and it is defined as: $\pi : \mathcal{O} \rightarrow \mathcal{P}(\mathcal{A})$. The
203 notation $a \sim \pi(\cdot \mid o)$ means that the action a is sampled from the distribution
204 induced by $\pi(\cdot \mid o)$. The notation $\pi(a \mid o)$ denotes the probability of selecting
205 action a under the distribution $\pi(\cdot \mid o)$.

1.2.3 Trajectory

Now that both the MDP and the policy π have been defined, we can characterize the interaction between them. A sequence of consecutive interactions between an environment and a policy is called a trajectory. A trajectory is generated through the repeated interaction between the policy and the environment, following the sequence below:

1. The environment, being in state s , provides an observation o to the policy π . Based on this observation, the policy outputs a probability distribution over actions $\pi(\cdot | o)$, from which an action is sampled: $a \sim \pi(\cdot | o)$.
2. The sampled action is executed in the environment. The environment then generates a transition: $(s', o', r) \sim p(\cdot | s, a)$. It moves to a new state s' , and returns a new observation o' and a reward r .

A trajectory t of length T can be written as:

$$t = (s_1, o_1, a_1, r_1, s_2, o_2, a_2, r_2, \dots, s_T, o_T, a_T, r_T).$$

It is important to clearly distinguish between a trajectory distribution and its realizations. We denote by τ the distribution over trajectories, and by t a particular trajectory. The distribution τ is determined by the initial state s , the initial observation o , the environment dynamics p , and the policy π , and is denoted by $\tau(s, o, p, \pi)$. The notation $t \sim \tau(s, o, p, \pi)$ means that the trajectory t is sampled from the distribution induced by s , o , p , and π .

An important property of the trajectory distribution is its recursive structure. Since both the policy π and the transition function p are known, the distribution over trajectories starting from s_1 with observation o_1 can be expressed in terms of the distribution over trajectories starting from the next state s_2 with observation o_2 . Specifically, the probability of a trajectory t factorizes as:

$$\tau(t | s_1, o_1, p, \pi) = \pi(a_1 | o_1) p(s_2, o_2, r_1 | s_1, a_1) \tau(t_2 | s_2, o_2, p, \pi),$$

where $t_2 = (s_2, o_2, a_2, r_2, \dots, s_T, o_T, a_T, r_T)$ denotes the trajectory starting from the second step.

There exist two special types of trajectories. First, a trajectory that terminates in a terminal state is called a terminal trajectory, and is denoted by $t_t \sim \tau_t(s, o, p, \pi)$. Second, a terminal trajectory that starts from an initial state and an initial observation is called an episode, and is denoted by $t_e \sim \tau_t(s^i, o, p, \pi)$.

1.2.4 Value Functions

To evaluate the ability of a policy π to collect rewards, we introduce the notion of return. The return of a trajectory $g(t, \gamma)$, is defined as the discounted sum of rewards collected along the trajectory:

$$g(t, \gamma) = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots + \gamma^{T-1} r_T = \sum_{i=1}^T \gamma^{i-1} r_i,$$

where $\gamma \in [0, 1]$ is the discount factor. This parameter controls the time horizon of the agent's objective. When γ is close to 0, the agent focuses almost exclusively

on immediate rewards, leading to short-sighted behavior. As γ increases toward 1, future rewards gain more weight, and the agent adopts a more far-sighted strategy. In theory, since the goal is to maximize long-term cumulative reward, we would like to set $\gamma = 1$. In practice, however, γ is often chosen slightly below 1 (e.g., 0.99) to stabilize learning, and to reflect the fact that distant future rewards are inherently more uncertain. Throughout this manuscript, unless explicitly stated otherwise, we assume $\gamma = 1$ to simplify notation.

The return also admits a recursive form, which is central to the dynamic programming and reinforcement learning algorithms presented later:

$$g(t, \gamma) = r_1 + \gamma g(t_2:),$$

From the notion of return, we can define functions that estimate the expected return of a policy from a given starting point. These functions are called value functions. The first one we examine in detail is the state-value function V . It tells us how desirable a particular state is under a given policy π , given an observation o and environment dynamics p . Strictly speaking, this function should be written as $V(s, o, p, \pi)$ to reflect all its dependencies. However, since the environment dynamics p are fixed throughout learning, we omit it from the notation. To emphasize that the value is tied to a specific policy π , we place π as a subscript. This yields the compact notation $V_\pi : \mathcal{S} \times \mathcal{O} \rightarrow \mathbb{R}$.

If $V_\pi(s, o) > V_\pi(s', o')$, then, under policy π , the agent prefers being in state s with observation o over state s' with observation o' . Formally, the state-value function is defined as:

$$V_\pi(s, o) = \mathbb{E}[g(t)], \quad \text{with } t \sim \tau_t(s, o, p, \pi).$$

It is worth noting that the state-value function V_π is recursive, a property that follows directly from the recursive form of the return. Using $g(t) = r_1 + \gamma g(t_2:)$, we can write:

$$\begin{aligned} V_\pi(s, o) &= \mathbb{E}[g(t)], \quad \text{with } t \sim \tau_t(s, o, p, \pi) \\ &= \mathbb{E}[r_1 + \gamma g(t_2:)] \\ &= \mathbb{E}[r_1 + \gamma V_\pi(s_2)], \end{aligned}$$

If we express the expectation explicitly in terms of the distributions p and π , we obtain the Bellman equation for V_π :

$$\begin{aligned} V_\pi(s, o) &= \mathbb{E}[r_1 + \gamma V_\pi(s_2)], \quad \text{with } t \sim \tau_t(s, o, p, \pi) \\ &= \sum_{a \in \mathcal{A}} \pi(a | o) \sum_{\substack{s' \in \mathcal{S} \\ o' \in \mathcal{O} \\ r \in \mathcal{R}}} p(s', o', r | s, a) [r + \gamma V_\pi(s', o')]. \end{aligned}$$

Now that the value of a state has been expressed, the next step is to consider the value of taking a specific action in a given state. Indeed, it is useful to estimate the expected future gain of an action a in state s , in order to compare different actions and determine which one is the most promising under a given policy π . This function is called the state-action value function $Q_\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$.

274 If $Q_\pi(s, a) > Q_\pi(s, a')$, then, under policy π , choosing action a in state s yields
 275 a higher expected return than choosing action a' . It is defined as:

$$Q_\pi(s, a) = \mathbb{E} \left[r + \gamma g(t) \right], \quad \text{with } t \sim \tau_t(s', o, p, \pi), \quad (s', o, r) \sim p(\cdot \mid s, a).$$

276 We can relate Q_π to V_π by recognizing that, after taking action a in state s ,
 277 the remaining return from the next state s' is precisely $V_\pi(s')$. This yields:

$$\begin{aligned} Q_\pi(s, a) &= \mathbb{E} \left[r + \gamma g(t) \right], \quad \text{with } t \sim \tau_t(s', o, p, \pi), \quad (s', o, r) \sim p(\cdot \mid s, a) \\ &= \mathbb{E} \left[r + \gamma V_\pi(s', o) \right] \\ &= \sum_{\substack{s' \in \mathcal{S} \\ o' \in \mathcal{O} \\ r \in \mathcal{R}}} p(s', o', r \mid s, a) \left[r + \gamma V_\pi(s', o) \right] \end{aligned}$$

278 1.2.5 Optimal Policy

279 The goal of reinforcement learning is to find a policy that maximizes, for every
 280 observation, the expected cumulative reward until a terminal state is reached.
 281 This policy is called the optimal policy and is denoted by π^* . Formally, π^* is a
 282 deterministic policy that always selects the action with the highest state-action
 283 value:

$$\pi^*(a \mid o) = \begin{cases} 1 & \text{if } a = \arg \max_{a' \in \mathcal{A}} Q_{\pi^*}(s, a'), \\ 0 & \text{otherwise,} \end{cases}$$

284 where s is the true state underlying the observation o .

285 It is worth noting that defining the optimal policy in a partially observable
 286 setting is more delicate than in the fully observable case. Since the policy only has
 287 access to observations o rather than the true state s , the observation must carry
 288 enough information to distinguish between states that require different optimal
 289 actions. For the sake of simplicity, we assume throughout this manuscript that
 290 this condition holds. This assumption is of interest when one seeks to establish
 291 mathematical proofs, but in real-world implementations it matters little.

292 As we have just seen, both V_π and Q_π can be expressed explicitly in terms
 293 of the environment dynamics p and the policy π . If p and π are fully known,
 294 one can theoretically compute the optimal policy by unfolding the graph of all
 295 possible trajectories starting from a given state s and observation o . This is
 296 possible precisely because V_π and Q_π are recursive functions. Solving an MDP
 297 in this way is known as dynamic programming.

298 In practice, most environments do have too many states to allow an explicit
 299 representation of the dynamics p . Storing and computing over the full transition
 300 function becomes intractable. To overcome this limitation, instead of directly
 301 constructing the optimal policy, we adopt an iterative approach: starting from
 302 an initial policy, we progressively improve it so that it converges toward π^* .
 303 This iterative improvement through interaction with the environment is precisely
 304 what constitutes learning in reinforcement learning, and it is the subject of the
 305 next section.

1.3 Learning

Now that the optimal policy π^* has been defined, we face the practical question of how to obtain it. The recursive Bellman equations for V_π and Q_π suggest dynamic programming approaches, but these scale poorly to large state spaces. Reinforcement learning offers an alternative: it seeks to solve the MDP iteratively, by generating a sequence of policies that progressively approach π^* through direct interaction with the environment. In this section, we introduce two major families of reinforcement learning methods used today: critic-only methods and actor-critic methods.

1.3.1 Critic-Only Methods

The central idea behind all critic-only approaches is to iteratively construct an approximation of the state-action value function Q_π , which we denote by $\hat{Q}_\pi$. Although $\hat{Q}_\pi$ is only an approximation of the true Q_π , the policy always acts greedily with respect to this approximation:

$$\pi(a \mid o) = \begin{cases} 1 & \text{if } a = \arg \max_{a' \in \mathcal{A}} \hat{Q}_\pi(o, a'), \\ 0 & \text{otherwise.} \end{cases}$$

It is precisely this way of modeling the policy that gives this family of methods its name. The policy is not represented explicitly as a standalone function. Rather, it simply emerges from searching over the set of actions to maximize $\hat{Q}_\pi$.

Since $\hat{Q}_\pi$ serves as the basis for the agent's decisions, it cannot rely on more information than the agent itself has access to. It must therefore be defined solely over observations, as a function $\hat{Q}_\pi : \mathcal{O} \times \mathcal{A} \rightarrow \mathbb{R}$. Its objective is to approximate the true Q_π , which depends on the underlying states s , using only the partial information available through o . Once again, this poses no issue in practice, as it reflects how these algorithms are actually deployed in real-world applications.

To give the reader a clear example of critic-only methods, we present them in their simplest form: model-free, without temporal-difference learning, without replay buffer, without bootstrapping, and using only ε -greedy exploration. Naturally, many variants and extensions of this basic scheme exist, but the core idea remains the same.

When implementing a reinforcement learning, one of the first questions that arises is how to represent the value function. Concretely, this means choosing a suitable model for $\hat{Q}_\pi$. Any machine learning model capable of handling a regression task can be used for this purpose. We denote the set of parameters of the model by θ . In deep reinforcement learning implementations, a neural network is used to model this function, in which case θ represents the weights and biases of the network.

In this setting, the action selection rule is modified as follows. With probability $1 - \varepsilon$, the policy selects the greedy action that maximizes $\hat{Q}_\pi$, and with probability ε it selects an action uniformly at random. The trajectory distribution induced by the resulting policy now depends on both π and ε , and is denoted by $t \sim \tau(s, o, p, \pi, \varepsilon)$. The parameter ε controls the degree of exploration. If the policy always chooses the action it currently believes to be optimal, it may miss

better alternatives that $\hat{Q}_\pi$ has not yet learned to value correctly. Exploring suboptimal actions from time to time is therefore essential to verify that $\hat{Q}_\pi$ accurately reflects the true values, while still focusing most of the time on the most promising actions. In more advanced implementations, ε can be adapted over the course of learning to control exploration dynamically, for instance by decreasing it gradually over time.

The learning process consists of two steps that repeat until convergence:

1. **Data collection.** The current policy π interacts with the MDP. For a given starting state s_1^i and observation o_1 , the interaction produces a episode t_e sampled from $\tau_e(s_1^i, o_1, p, \pi, \varepsilon)$. For each step i along the episode, the return from that step onward is $g(t_{i:}) = \sum_{k=i}^T \gamma^{k-i} r_k$, which serves as the target value for the state-action pair (s_i, a_i) .
2. **Model update.** The tuples $(s_i, a_i, g(t_{i:}))$ obtained in the previous step serve as supervised training data: the state-action pair (s_i, a_i) is the input, and the return $g(t_{i:})$ is the associated target. These examples are used to train $\hat{Q}_\theta$. We denote the improved approximation by $\hat{Q}'_\theta$, which now predicts returns more accurately under π . Since $\hat{Q}'_\theta$ has changed, the policy derived from it also changes. we denote this new policy by π' .

Because the policy is now π' , the approximation $\hat{Q}'_\theta$ must be trained to predict returns under π' . This requires returning to step 1 to collect fresh interaction data generated by π' . The two steps thus alternate until the policy and the value approximation stabilize.

Even this simple algorithm already illustrates a difficulty inherent to all forms of reinforcement learning: the instability of the learning process. Each update from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ causes the policy to shift from π to π' . For the learning to remain stable, the change from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ should be as small as possible. This way, $\hat{Q}'_\theta$, trained to predict returns under π , remains reasonably accurate under π' . However, the smaller the change, the slower the learning progresses. A trade-off must therefore be struck between stability and learning speed. Many improvements introduced later in the literature aim at stabilizing this learning loop [?].

Convergence guarantees for such algorithms have been proved under a different set of assumptions than the setting presented here. For example, in the classical convergence results, the MDP is fully observable, the value function is stored in a table, and the learning rate is decreased at a suitable rate over time [?]. In modern implementations, the value function is approximated by a neural network. Theoretical convergence proofs no longer hold in this setting. Nevertheless, deep methods have demonstrated remarkable empirical success, making them the tool of choice for RL tasks.

Critic-only approaches suffer from several limitations. The main limitation is scalability to large action spaces. Since the policy must evaluate all possible actions at each decision step to select the maximum, the computational cost grows with the size of the action set. To address this issue, another family of reinforcement learning algorithms exists: actor-critic methods.

1.3.2 Actor-Critic Methods

Two principal modifications distinguish actor-critic methods from critic-only approaches. First, instead of approximating the state-action value function Q , we now aim to approximate the state-value function V . Second, the policy π is given its own dedicated set of parameters. We thus have two separate components: a value function approximation $\hat{V}_{\theta_1}$ and a parameterized policy π_{θ_2} .

Analogously to critic-only methods, $\hat{V}_{\theta_1}$ is trained using supervised learning on pairs obtained from interaction with the MDP. For each step i along a trajectory, the input is the state-observation pair (s_i, o_i) and the target is the observed return $g(t_i)$. The value function is thus trained to correct its predictions based on data collected by the current policy interacting with the environment.

The key difference lies in how the policy π itself is optimized. Since the policy now has its own set of parameters θ_2 , it can be updated directly. The update uses the notion of advantage, defined as:

$$A(s, a) = g(t) - \hat{V}_{\theta_1}(s, o),$$

where t is the trajectory that starts by taking action a in state s and then follows π for all subsequent steps.

When $A(s, a) > 0$, the action a performed better than expected, and the policy parameters θ_2 are adjusted to increase the probability $\pi_{\theta_2}(a | o)$. When $A(s, a) < 0$, the action a performed worse than expected, and the probability is decreased.

Both $\hat{V}_{\theta_1}$ and π_{θ_2} are improved simultaneously within the same loop: the critic provides a learned baseline for the actor, and the actor generates the data used to train the critic. This interplay reduces the instability inherent to reinforcement learning.

PPO (Proximal Policy Optimization), which is the current standard in reinforcement learning, belongs to this family of actor-critic methods. It will be used throughout this manuscript for all reinforcement learning experiments.

Now that we are capable of training highly performant policies, which learn purely through interaction with their environment without any human supervision, the question arises of how to explain their behavior. This is the subject of the next chapter: explainability in the context of reinforcement learning.